



Merging Parameterized Task Graphs in PaRSEC Through JDF Composition with LLM Assistance

Zhuowei Gu^{1,4}, Aurelien Bouteiller^{2,4}, Joseph Schuchart^{3,4}, Thomas Herault^{1,4},
and Qinglei Cao^{1,4}(✉)

¹ Saint Louis University, St. Louis, USA

² Advanced Micro Devices, Santa Clara, USA

³ Stony Brook University, Stony Brook, USA

⁴ Inria, Paris, France

Abstract. The Parameterized Task Graph (PTG) interface in the PaRSEC runtime system provides a powerful mechanism to express scalable, parameterized task dependencies for distributed heterogeneous computing. However, as applications become increasingly complex and hierarchical, developers often need to compose or merge multiple JDF (Job Data Flow) descriptions—each representing a subgraph—into a single coherent PTG, a process that is currently manual, error-prone, and difficult to generalize. In this work, we introduce a Python- and AI-assisted tool to automate the merge-to-JDF process in PaRSEC. The tool recursively analyzes two or more JDFs, identifies overlapping dataflows, and merges their task and data definitions into a unified PTG while preserving correctness and dependency semantics. We demonstrate this approach on representative PTG applications, including multi-stage solvers and AI model, like Transformer, showing that the automated merge tool substantially reduces development effort while maintaining runtime efficiency.

Keywords: High-performance computing · Task-based runtime · Parameterized task graph · Task graph composition

1 Introduction

Task-based runtime systems provide a high-level programming model in which applications are expressed as collections of tasks with explicit data dependencies, allowing the runtime to dynamically schedule and execute tasks while exposing fine-grained parallelism. Such models have become a cornerstone for achieving scalability and performance on modern high-performance computing (HPC) platforms [1–5]. PaRSEC [4, 5] is a general-purpose task-based runtime system designed to efficiently execute large-scale parallel applications on distributed and heterogeneous architectures. To support scalable task representation and

scheduling, ParSEC adopts its domain-specific languages (DSLs), e.g., Parameterized Task Graph (PTG) [4, 6], as its core abstraction. PTG compactly represents families of directed acyclic graphs (DAGs) by expressing computation as parameterized task classes with explicit tile-level read/write dependencies, enabling the runtime to dynamically discover, schedule, and execute massive numbers of fine-grained tasks without materializing the full DAG, while preserving correctness and a high degree of concurrency.

The PTG/JDF (Job Data Flow) model has proven particularly efficient and scalable for structured numerical algorithms and increasingly for emerging AI workloads [7–12], where applications naturally decompose into repeated computational kernels such as tiled Cholesky factorization [13, 14], General Matrix Multiplication (GEMM) [15, 16] and Multi-Head Attention (MHA). However, as applications grow in complexity, PTGs are rarely authored as a single monolithic graph. Instead, developers tend to express applications as a collection of smaller PTGs or JDFs, each implementing a logical kernel or algorithmic stage. For instance, AI models like Transformer, each sub-layer can be written as a kernel in PTG such as linear projection, reshape, dot-production, and softmax. In the absence of PTG-level merging, composing multiple PTGs inherently results in a bulk-synchronous parallel (BSP) model, even when each individual PTG exposes fine-grained parallelism.

Merging multiple DAGs into a fused DAG raises a major usability challenge. In practice, it is uncommon to develop a single JDF for an entire application due to the high maintenance cost. This issue is well illustrated by the example of solving a linear algebra problem, which naturally decomposes into a sequence of reusable kernels. While each individual JDF can be implemented and optimized independently as a DAG, assembling them into a complete workflow still requires users to manually connect the dataflows across JDFs. Manually merging JDFs is not user friendly in runtime system.

PTG composition can in principle be addressed at multiple layers. A DSL/compiler approach may simplify algorithm expression, but it requires compiler changes; and when cross-PTG composition is needed, users still must generate/merge JDFs and recompile. Runtime-level composition is even more challenging, as it requires dynamically reconciling dependency semantics across independently defined graphs while preserving determinism, scalability, and low overhead. In contrast, performing composition at the PTG/JDF level frames the problem as a compile-time graph transformation: multiple correct subgraphs are merged into a unified PTG by explicitly connecting their data flows before execution.

Motivated by these limitations, we introduce a Python-based, AI-assisted tool for PTG composition in ParSEC that treats JDF merging as a compile-time graph transformation. The tool analyzes multiple JDFs, identifies compatible dataflows, and generates a unified PTG by explicitly connecting cross-graph dependencies prior to execution. By supporting recursive composition—i.e., the iterative merging of two or more JDF subgraphs—it enables fine-grained integration of reusable PTG building blocks while preserving asynchronous execu-

tion and avoiding coarse-grain synchronization barriers. This approach reduces developer effort, eliminates fragile manual scripting, and provides a scalable and user-friendly mechanism for assembling complex HPC and AI workflows without requiring runtime modifications.

The remainder of this paper is organized as follows. Section 2 reviews related work on task-based runtimes and task graph composition. Section 3 introduces PaRSEC runtime system and JDF structure. Section 4 introduces Python- and AI-assisted tool for PTG composition. Section 5 evaluates the performance impact of PTG fusion on representative systems. Finally, Sect. 6 concludes the paper and discusses directions for future work.

2 Related Work

Modern task-based runtime systems allow applications to express concurrency and data dependencies while delegating scheduling to the runtime. In sequential task-flow (STF) systems such as StarPU [1], OmpSs [17], OpenMP [2], HPX [18], and PaRSEC DTD [19], tasks are dynamically inserted at runtime (e.g., via `insert_task` or task directives) with data access annotations that enable on-the-fly dependency inference. The global task graph is therefore constructed incrementally during execution rather than being statically available. While this model effectively exposes fine-grained parallelism within a single operation, composition across independently defined DAGs is constrained by sequential submission and runtime dependency resolution.

Graph-centric systems such as Template Task Graphs (TTG) [20] provide explicit graph composition mechanisms at the C++ API level. TTG enables compile-time composition by connecting input and output terminals of graph objects before runtime instantiation. Similarly, HPX supports task composition using futures and continuations to construct higher-level execution graphs prior to scheduling. However, in both cases, composition occurs at the C++ programming layer rather than through structural rewriting of a task-graph DSL.

In contrast, PaRSEC’s PTG interface expresses task graphs symbolically in the JDF language, which is compiled into a taskpool before execution. While PaRSEC supports DTD, it does not provide a native mechanism for statically merging independently authored PTGs at the JDF level. We treat PTG composition as a compile-time graph transformation performed before JDF compilation, enabling structural merging of independent PTGs while preserving asynchronous execution semantics.

3 PaRSEC Runtime System

PaRSEC [5, 21] is a task-based runtime system designed for asynchronous and architecture-aware execution on distributed, many-core, and heterogeneous platforms. It executes applications by dynamically instantiating tasks from a compact graph description and scheduling them across available resources while enforcing data dependencies. PaRSEC transparently manages data movement

across memory spaces, including transfers between nodes and between different memory hierarchies on heterogeneous devices, enabling efficient execution without exposing low-level details to the user. To further improve programmability, PaRSEC provides multiple DSLs [4], such as PTG and DTD [19], which allow domain scientists to express application logic at a high level while abstracting away hardware complexity, memory hierarchy, and communication mechanisms. In the PTG model, application logic is written in the JDF language and then compiled into C source files by the PaRSEC PTG compiler, which generates a taskpool implementation capturing the symbolic, parameterized task dependencies. The generated C code is compiled and linked with the PaRSEC library to form an executable, and at runtime PaRSEC uses the encoded symbolic DAG description to dynamically discover and schedule task instances based on data availability across distributed resources. This paper focuses on task-graph merging for PTG, whose efficiency and scalability have been demonstrated across a range of applications, including weather forecasting [11, 14], climate emulation [9], genomics [8, 22], etc.

4 Methodology

Merging task graphs is particularly challenging in runtime system because these scripts vary widely in structure and coding style across developers and projects, making merge logic difficult to reuse, validate, and maintain. This motivates our approach in PaRSEC: rather than reconciling heterogeneous C composition code, we perform PTG composition directly at the JDF level. Since adequate training samples were initially unavailable, we first developed a deterministic Python script to procedurally generate correct fused JDFs. The outputs of this script served as the ground truth for our AI model, ensuring the trained model strictly adheres to the established merging principles.

4.1 Python-Based JDF Merge Procedure

Table 1. Core structural elements of PTG/JDF relevant to merging

Component	Role in PTG/JDF and Merge Context
Task Classes	Parameterized task templates defining families of task instances.
Execution Spaces	Index domains that determine valid task instances.
Dataflows	READ/WRITE specifications encoding dependency semantics.
Executable Bodies	Computation kernels associated with task classes.
Wrapper Code	Initialization and submission logic for taskpools.

With the PTG/JDF structure introduced in Table 1, we now describe how our Python script merges multiple independent JDFs into a single fused JDF.

The merge procedure follows the same structural decomposition of a JDF and rewrites only the components that determine task dependencies and symbol consistency, allowing new users to compose PTGs without requiring detailed knowledge of JDF syntax or algorithm-specific logic.

a) Task Classes: The script first collects all task classes from the base JDF and the additional JDFs. To avoid name collisions, each non-base JDF is assigned a unique prefix, which is applied consistently to task class names. Task classes are treated as the primary units of composition, and the fused JDF is formed by emitting the union of renamed task classes together with newly introduced cross-JDF dependencies.

b) Execution Spaces: For each task class, the script extracts its execution space parameters, including index variables, ranges, and inline expressions. These execution spaces are preserved in the fused JDF. When connecting task classes from different JDFs, the script establishes explicit mappings between execution space indices, supporting both forward and reverse index expressions.

c) Dataflows: Dataflow declarations encode READ, WRITE, and READ-WRITE accesses and fully determine task dependencies in a PTG. The script analyzes these dataflows to identify data dependency. During merging, dataflow clauses are rewritten to introduce explicit cross-JDF dependencies, replacing descriptor-based reads with direct task-to-task edges when appropriate.

d) Executable Bodies: Task bodies are preserved verbatim during the merge process. All BODY blocks, including device-specialized variants such as BODY [type = CUDA], are retained. Only symbol references affected by renaming are updated, ensuring that computational kernels and device mappings remain unchanged in the fused PTG.

e) Wrapper Code: Finally, the script integrates the wrapper code associated with each input JDF. All user-defined helper routines (e.g., workspace creation, CUDA handle initialization, and other auxiliary utilities) are preserved and merged as needed. Header files are consolidated, and a single copyright block is retained. In addition, the script generates a unified wrapper interface for the fused PTG, consisting of (i) a taskpool construction function that instantiates the fused task graph and initializes runtime metadata, (ii) a corresponding destruction function that releases taskpool resources, and (iii) a wrapper function that enqueues the taskpool into a ParSEC context and drives its execution to completion. Together, these functions allow the fused PTG to be invoked as a single, self-contained taskpool without requiring users to understand the internal structure of the merged JDF.

While the Python-based script provides a systematic way to merge JDF-level dependencies, merging wrapper code is significantly more challenging due to diverse C coding styles and application-specific helper routines. This motivates the use of an AI-assisted tool to handle complex and irregular merge cases, which we describe next.

4.2 AI-Assisted Tool Procedure

We extend our workflow with an AI-assisted tool for PTG composition. To construct this AI-assisted tool, we evaluate models of different parameter scales and explore both prompt-based generation and supervised fine-tuning to align model outputs with PTG/JDF structural semantics. We select the Qwen [23] series as the base model due to its publicly available weights, strong code-generation capability, and support for controlled fine-tuning. In contrast, recent proprietary large models are not openly accessible for parameter-level adaptation, which limits reproducibility and prevents direct integration into our training pipeline.

During inference, we further adopt a retrieval-augmented generation (RAG) strategy, where the model is provided with relevant reference materials—such as representative JDF examples, common compiler errors, common header files, JDF structure, and structural JDF syntax cheat-sheet—to guide generation. This retrieval step constrains the model’s reasoning within the PTG/JDF specification space and improves structural consistency without modifying model parameters.

a) Qwen2.5-Coder-14B-Base: We fine-tune Qwen2.5-Coder-14B-Base using 191 structured training samples collected from existing ParSEC and DPLASMA JDF programs. Each sample consists of a structured PTG specification—capturing global declarations, task classes, execution spaces, and dataflow relations—paired with its corresponding complete JDF implementation. Fine-tuning is performed via supervised instruction tuning (SFT) using a chat-style format, where only the assistant-generated JDF content contributes to the training loss. During inference, the fine-tuned model operates under a retrieval-grounded and structured prompting setup. The input prompt includes the PTG specification together with curated reference materials. The structured prompt explicitly specifies merge constraints and generation requirements, enabling the model to produce full JDF programs directly from PTG descriptions while preserving structural semantics.

b) Qwen3.0-Coder-Next: For larger-scale foundation models, we adopt a parameter-free inference strategy without fine-tuning. Specifically, Qwen3.0-Coder-Next (80B) is used in conjunction with retrieval-augmented generation (RAG), where the input prompt includes both the structured PTG specification and curated reference materials, such as JDF templates, structural rules, and representative examples. This approach enables structural reasoning over PTG composition while preserving the original model parameters. We didn’t fine tune Qwen3.0 because the existing sample size is too small.

c) GPT-5.2: Since GPT-5.2 is not publicly available for parameter-level fine-tuning, we employ it as an interactive inference engine without model adaptation. PTG specifications and merge objectives are provided through a structured prompt, and generation is guided via RAG, where relevant project-specific materials—such as ParSEC tutorials, representative JDF examples, and deterministic merge rules—are injected into the prompt context. The interaction is conducted iteratively within a chat-based environment, enabling continuous rea-

soning and refinement while constraining outputs to comply with PTG/JDF structural semantics.

5 Performance Results and Analysis

5.1 Experimental Settings

The experiments are conducted on two high-performance computing systems:

- **Innovative Computing Laboratory(ICL) Cluster:** A cluster maintained by the ICL at the University of Tennessee. The experiments are conducted on GPU-enabled nodes powered by Intel(R) Xeon(R) Gold 6140 CPUs, including systems equipped with NVIDIA A100-SXM4-80G, NVIDIA H100 PCIe-81.6GB, Tesla V100-SXM2-32GB, and NVIDIA A100-PCIE-40GB. These nodes are primarily used for development-scale experiments and detailed kernel level performance analysis.

- **Polaris:** A GPU-accelerated supercomputer at the Argonne Leadership Computing Facility, comprising 560 compute nodes. Each node integrates one AMD EPYC 7543P 32-Core “Milan” processor alongside four NVIDIA A100-SXM4-40GB. The system is connected via an HPE Slingshot-11 interconnect and is designed to support large-scale simulations and AI workloads.

On the ICL cluster, experiments target two distinct configurations: a single NVIDIA A100-PCIE-40GB GPU, and a dense node equipped with eight Tesla V100-SXM2-32GB GPUs. In both cases, a single MPI rank is used per node. On Polaris, experiments are scaled across either one or two compute nodes, utilizing one MPI rank per node to drive the four on-node A100 GPUs.

5.2 Experimental Examples

To evaluate the correctness and performance of the proposed PTG merge framework, we select two representative workloads, both contain multiple stages of kernel invocations with explicit data dependencies, making them suitable for evaluating fine-grain PTG composition.

a) Tiled Cholesky Factorization: Classical Cholesky factorization is typically expressed over scalar matrix elements with nested loops, resulting in relatively coarse-grained dependencies. In contrast, the tiled variant partitions a symmetric positive-definite matrix into square tiles and performs computation at tile granularity, exposing fine-grain parallelism. The upper-triangular tiled pipeline is composed of four BLAS-based kernels—POTRF, TRSM, SYRK, and GEMM [24]—where each tile-level kernel invocation forms a task instance with explicit data dependencies. This formulation induces a substantially richer and more concurrent task graph than the classical version, with task count scaling as $\mathcal{O}((N/b)^3)$. The non-fused and fused PTG workflows are illustrated in Fig. 1, making tiled Cholesky a representative structured benchmark for evaluating recursive PTG composition. (LUT: Left upper triangular conjugate transpose, LUN: Left upper triangular non-transpose).

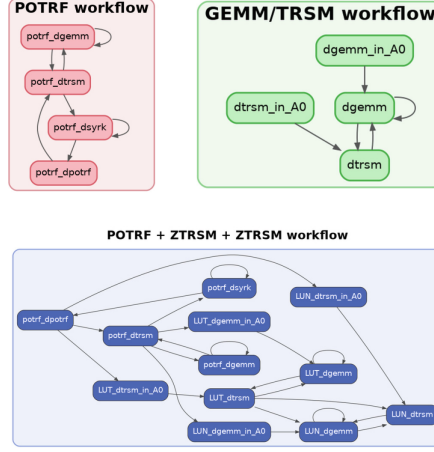


Fig. 1. PTG structure of the upper-triangular Cholesky factorization workflow before and after fusion. TOP: independently defined POTRF and GEMM/TRSM task graphs. DOWN: fused PTG produced by the proposed merge framework.

b) MHA: MHA represents a modern AI workload composed of multiple computational stages forming a deep pipeline. A typical MHA layer includes linear projections for query, key, and value tensors, tensor reshaping and transposition operations, scaled dot-product attention (including matrix multiplication and softmax), and a final output projection. Each stage can be expressed as a kernel-level PTG, such as GEMM-based linear transformations and batched matrix multiplications.

5.3 Correctness and Structural Accuracy of PTG Fusion

To evaluate the structural correctness of AI-assisted PTG fusion, we compare the fused JDF programs generated by different models against manually verified ground-truth implementations. All models are evaluated under identical prompts and the same retrieval-augmented generation (RAG) setup to ensure a fair comparison.

Textual similarity between generated JDFs and the reference implementation is measured using an online similarity tool¹, which computes token-level structural similarity. The resulting similarity scores are summarized in Table 2.

The results indicate a substantial gap in full-structure generation accuracy. While GPT-5.2 produces outputs that are largely consistent with the reference fused JDF (and require only minor manual adjustments before direct use), both Qwen2.5 and Qwen3.0 exhibit low global similarity, suggesting instability in long-range, constraint-preserving code generation.

¹ <http://life.chacuo.net/convertsimilar>.

Table 2. Text similarity between generated fused JDFs and ground truth under different training and inference configurations (including wrapper generation).

Model	Fine-tuned	Inference Strategy	Similarity
Qwen2.5-Coder-14B	Yes	Structured Prompt + RAG	8.62%
Qwen3.0-Coder-Next	No	Structured Prompt + RAG	15.95%
GPT-5.2	No	RAG (minimal prompt)	75.53%
GPT-5.2	No	Structured Prompt + RAG	96.62%

However, a deeper structural analysis reveals an important nuance. Although Qwen3.0 performs poorly in full-program generation, its structural understanding of PTG components remains strong. Table 3 summarizes structural extraction accuracy for Qwen3.0.

Table 3. Structural extraction accuracy of Qwen3.0 in 191 samples

Component	Correct / Total	Accuracy
Globals fully correct (per sample)	170 / 191	89%
Task names fully correct (per sample)	186 / 191	97%
Tasks fully correct (per sample)	154 / 191	81%
Parameters fully correct (per sample)	189 / 191	99%
Execution space fully correct (per sample)	165 / 191	86%
Body blocks fully correct (per sample)	189 / 191	99%

These results suggest that Qwen3.0 demonstrates high accuracy in structural parsing and local component extraction (e.g., task names, parameters, body blocks), but struggles in synthesizing globally consistent fused JDF programs. In particular, merge tasks require long-range constraint tracking across headers, dataflow rewrites, wrapper adaptation, and cross-graph task mapping. Small deviations in descriptor naming or dataflow rewriting can propagate globally and significantly reduce textual similarity.

In contrast, GPT-5.2 shows substantially stronger long-form structural generation stability. The generated fused JDF programs not only maintain correct task dependencies and execution spaces but also successfully integrate complex wrapper-level logic. In most evaluated cases, GPT-5.2 outputs are directly usable after minor manual refinement.

Overall, the results indicate that the primary limitation of open-weight models in PTG fusion is not syntax understanding but constraint-preserving structured generation. This distinction is critical when evaluating AI-assisted graph composition for DSL-level program synthesis.

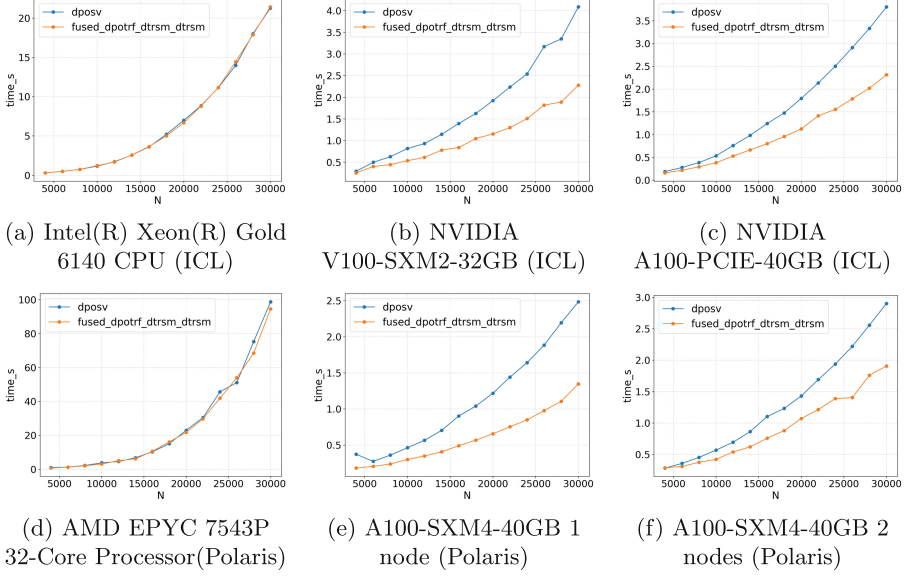


Fig. 2. Performance comparison between unmerged and fused JDFs.

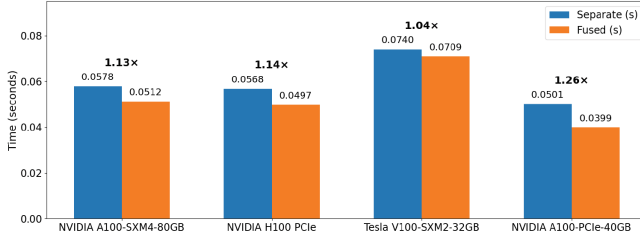


Fig. 3. Performance result of merging Transformer sublayer(Linear Projection + Reshape).

5.4 Performance Evaluation

We evaluate the impact of PTG fusion on two representative workloads with different structural properties. The tiled Cholesky factorization represents a recursively composed multi-stage pipeline, whereas the Transformer MHA sublayer corresponds to a lighter-weight, single-stage PTG merge between adjacent kernels.

For tiled Cholesky factorization, the performance results are shown in Figs. 2(a)–(f). The matrix size is up to 30,000 in this experiment. The tile size is still in the paper, 1024. On GPU-accelerated platforms, the fused implementation consistently outperforms the unfused version. This improvement primarily stems from reduced data movement and lower runtime overhead. By merging factorization and update kernels, intermediate data remains resident in GPU

memory, eliminating redundant Host-to-Device (H2D) transfers over PCIe and reducing synchronization between separate PTGs. For example, on a single A100 system (Fig. 2(c)) at the largest matrix size, the H2D data volume decreases from 11.11 GB to 8.00 GB, corresponding to a reduction of 28%. Fusion also reduces the number of tasks in the execution DAG, leading to coarser granularity and a more compact execution pipeline, which is particularly beneficial on GPUs where kernel launch latency and scheduling overhead are significant. In contrast, on CPU-only systems (Figs. 2(a) and (d)), the fused and unfused versions exhibit nearly identical performance. Since CPU cores share main memory, there is no device-level transfer overhead to eliminate, and execution time remains dominated by computation rather than scheduling costs.

For the Transformer MHA sub-layer, performance results are shown in Fig. 3. Here, fusion is applied between the linear projection and reshape stages, eliminating intermediate task boundaries. Across different GPU architectures, the fused implementation achieves speedups ranging from $1.04\times$ to $1.26\times$. Although this workload does not involve recursive multi-stage composition like Cholesky, it is more sensitive to kernel launch overhead and memory traffic. By merging adjacent PTGs into a unified execution graph, the fused version reduces synchronization and improves execution locality, demonstrating that even single-stage PTG fusion can yield measurable performance gains in AI-assisted tool.

6 Conclusion and Future Work

This work introduces a Python-based JDF merge framework for PaRSEC, augmented by an AI-assisted approach to handle complex and irregular fusion scenarios. By treating PTG composition as a compile-time graph transformation, our method enables structured merging of independently developed JDFs while preserving task dependencies and dataflow semantics. Experiments on tiled Cholesky factorization and Transformer MHA workloads validate both structural correctness and practical applicability of the approach.

Our results demonstrate that deterministic JDF rewriting reliably preserves semantic correctness for well-structured PTG compositions, while AI-assisted generation provides flexibility in handling wrapper integration and non-canonical dataflows. Among the evaluated models, GPT-5.2 achieves the highest structural consistency under structured prompting, whereas open-weight models show strong local structural understanding but weaker global constraint preservation.

Performance evaluation shows that PTG fusion reduces data movement and runtime overhead on GPU-accelerated systems, yielding measurable speedups for both HPC and AI workloads. Future work will focus on integrating performance-aware fusion heuristics and developing training strategies that improve long-range constraint reasoning for structured DSL generation.

Acknowledgments. This research was supported by the U.S. National Science Foundation under Award OAC-2451577. For computational resources, this work utilized the compute node at the Innovative Computing Laboratory of the University of Tennessee,

Knoxville. This research also used resources (Polaris cluster) of the Argonne Leadership Computing Facility, which is a U.S. Department of Energy Office of Science User Facility operated under contract DE-AC02-06CH11357.

References

1. Augonnet, C., Thibault, S., Namyst, R.: StarPU: a unified runtime system for heterogeneous multicore architectures. In: *Proceedings of the 18th International Conference on Parallel Architectures and Compilation Techniques*, pp. 1–10. IEEE (2009). StarPU builds and schedules task dependency graphs dynamically at runtime
2. OpenMP. OpenMP 4.5 Complete Specifications (2015)
3. Duran, A., Ferrer, R., Ayguade, E., Badia, R.M., Labarta, J.: A proposal to extend the OpenMP tasking model with dependent tasks. *Int. J. Parallel Program.* **37**(3), 292–305 (2009)
4. Bosilca, G., Bouteiller, A., Danalis, A., Faverge, M., Hérault, T., Dongarra, J.J.: Parsec: exploiting heterogeneity to enhance scalability. *Comput. Sci. Eng.* **15**(6), 36–45 (2013)
5. Bouteiller, A., Hérault, T., Cao, Q., Schuchart, J., Bosilca, G.: PaRSEC: scalability, flexibility, and hybrid architecture support for task-based applications in ECP. *Int. J. High Performance Comput. Appl.* (2024)
6. Danalis, A., Bosilca, G., Bouteiller, A., Hérault, T., Dongarra, J.: PTG: an abstraction for unhindered parallelism. In: *2014 Fourth International Workshop on Domain-Specific Languages and High-Level Frameworks for High Performance Computing*, pp. 21–30. IEEE (2014)
7. Alomairy, R., Abdulah, S., Cao, Q., Genton, M.G., Keyes, D.E., Ltaief, H.: Sustainably modeling a sustainable future climate. In: *2025 IEEE High Performance Extreme Computing Conference (HPEC)*, pp. 1–8. IEEE (2025)
8. Alomairy, R., Cao, Q., Ltaief, H., Keyes, D., Edelman, A.: Scalable hamming distance computation using accelerated matrix transformations. In: *ISC High Performance 2025 Research Paper Proceedings (40th International Conference)*, pp. 1–13. Prometheus GmbH (2025)
9. Abdulah, S., et al.: Boosting earth system model outputs and saving petabytes in their storage using exascale climate emulators. *ACM Gordon Bell Prize for Climate Modelling Finalist* (2024)
10. Cao, Q., Abdulah, S., Ltaief, H., Genton, M.G., Keyes, D., Bosilca, G.: Reducing data motion and energy consumption of geospatial modeling applications using automated precision conversion. In: *2023 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 330–342. IEEE (2023)
11. Cao, Q., et al.: Reshaping geostatistical modeling and prediction for extreme-scale environmental applications. In: *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis (ACM Gordon Bell Prize Finalist)*, pp. 1–12. IEEE (2022)
12. Cao, Q., et al.: A framework to exploit data sparsity in tile low-rank cholesky factorization. In: *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 414–424. IEEE (2022)
13. Cao, Q., et al.: Performance analysis of tile low-rank cholesky factorization using parsec instrumentation tools. In: *2019 IEEE/ACM International Workshop on Programming and Performance Visualization Tools (ProTools)*, pp. 25–32. IEEE (2019)

14. Cao, Q., et al.: Extreme-scale task-based cholesky factorization toward climate and weather prediction applications. In: Proceedings of the Platform for Advanced Scientific Computing Conference, pp. 1–11 (2020)
15. Zhang, Q., Alomairy, R., Wang, D., Zhuowei, G., Cao, Q.: Leveraging Hardware-Aware Computation in Mixed-Precision Matrix Multiply: A Tile-Centric Approach. In: Diehl, P., Cao, Q., Herault, T., Bosilca, G. (eds.) Workshop on Asynchronous Many-Task Systems and Applications. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-97196-9_15
16. Zhang, Q., Alomairy, R., Wang, D., Zhuowei, G., Cao, Q.: High-performance mixed-precision matrix multiplication via tile-centric design on modern architectures. *SN Comput. Sci.* **7**(1), 24 (2026)
17. Barcelona Supercomputing Center. OmpSs: a task-based parallel programming model. <https://pm.bsc.es/ompss> (2026). OmpSs with Nanos++ runtime builds dependency graphs at execution time from annotated tasks
18. Heller, T., Kaiser, H., Iglberger, K.: Application of the ParalleX execution model to stencil-based problems. *Comput. Sci. Res. Dev.* **28**(2–3), 253–261 (2013)
19. Hoque, R., Herault, T., Bosilca, G., Dongarra, J.: Dynamic Task Discovery in PaRSEC: A Data-flow Task-based Runtime. In: Proceedings of the 8th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems, Scala '17 (2017)
20. Bosilca, G., Bouteiller, A., Herault, T., Dongarra, J.: Composition of algorithmic building blocks in template task graphs. In: Proceedings of the 2022 IEEE/ACM Parallel Applications Workshop: Alternatives To MPI+X (PAW-ATM), pp. 1–10. IEEE (2022)
21. Bosilca, G., Bouteiller, A., Danalis, A., Faverge, M., Herault, T., Dongarra, J.: PaRSEC: a programming paradigm exploiting heterogeneity for enhancing scalability. *Comput. Sci. Eng.* **99**, 1 (2013)
22. Ltaief, H., et al.: Toward capturing genetic epistasis from multivariate genome-wide association studies using mixed-precision kernel ridge regression. ACM Gordon Bell Prize Finalist (2024)
23. Yang, A., et al.: Qwen2.5 technical report. arXiv preprint [arXiv:2412.15115](https://arxiv.org/abs/2412.15115) (2024)
24. Lawson, C.L., Hanson, R.J., Kincaid, D.R., Krogh, F.T.: Basic linear algebra subprograms for fortran usage. *ACM Trans. Math. Software* **5**(3), 308–323 (1979)